Project: CI/CD for Cloud Native

Due: 04/04/2026 11:59 PM
Extended Deadline: 04/07/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Tekton CI Pipeline
- Task Overview
- Task

Flux CD Pipeline
- Task Overview
- Task

Wrap Up
- Conclusion

Flux CD Pipeline (Task)

A Fully-Automated Build

In the last task, we focused on building our Continuous Integration pipeline. We now will focus on connecting this pipeline to our Continuous Deployment Pipeline we built in the last project.

Intro. to Flux

Flux is an open-source GitOps tool for Kubernetes that keeps your cluster in sync with a Git repository. Rather than manually running `kubectl apply` every time you want to deploy a change, you commit your Kubernetes manifests to a repo and Flux continuously watches that repo, automatically applying any changes to your cluster. This means Git becomes the single source of truth for your infrastructure — every deployment is traceable, reversible, and auditable through your commit history.

In this task, Flux plays two roles. First, it manages the lifecycle of your Tekton CI pipeline resources by watching the `cipipeline/` directory in your repo and keeping those manifests applied to the cluster. Second, it handles automatic image promotion through its Image Automation controllers — when your Tekton pipeline builds and pushes a new image to ACR, Flux detects the new tag, updates the image reference in your deployment manifests, and commits that change back to Git, completing the CI/CD loop without any manual intervention.

Install the Flux CLI

The Flux CLI is how you interact with Flux from your terminal — checking status, triggering reconciliations, and debugging issues. You need it installed before Terraform can successfully bootstrap Flux into your cluster.

Linux:

```
curl -s https://fluxcd.io/install.sh |
```

Verify the installation:

```
flux --version
```

Here are the key CLI commands you'll use throughout this task:

Command	What it does
`flux check`	Verifies the cluster meets Flux's requirements
`flux get kustomizations`	Lists all Kustomizations and their sync status
`flux get sources git`	Lists all GitRepository sources
`flux get image repository`	Lists all ImageRepositories
`flux get image policy`	Lists all ImagePolicies
`flux reconcile source git flux-system`	Forces an immediate sync from your Git repo
`flux logs`	Streams logs from all Flux controllers
`flux events`	Shows recent Flux events across the cluster

Add Flux Bootstrapping to Terraform

You'll need to extend `main.tf` to also install and configure Flux. This requires adding the `flux` and `kubernetes` providers alongside the existing `azurerm` and `github` ones.

Step 1 — Add the new providers to your `required_providers` block:

```
flux = {
  source  = "fluxcd/flux"
  version = "~> 1.3"
}
kubernetes = {
  source  = "hashicorp/kubernetes"
  version = "~> 2.0"
}
```

Your `terraform` block should now look like this:

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    github = {
      source  = "integrations/github"
      version = "6.9.0"
    }
    flux = {
      source  = "fluxcd/flux"
      version = "~> 1.3"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
  }
}
```

Step 2 — Add provider configurations for `kubernetes` and `flux`:

These go after your existing `provider "github"` block. They reference the AKS cluster you're already creating, so the cluster must exist before Flux can be bootstrapped — Terraform handles this ordering

automatically via the `depends_on` you'll add in the next step.

```
provider "kubernetes" {
  host                   = azurerm_kube
  client_certificate     = base64decode
  client_key             = base64decode
  cluster_ca_certificate = base64decode
}

provider "flux" {
  kubernetes = {
    host                   = azurerm_ku
    client_certificate     = base64deco
    client_key             = base64deco
    cluster_ca_certificate = base64deco
  }
  git = {
    url = "https://github.com/${var.gh_
    http = {
      username = var.gh_org
      password = var.gh_token
    }
    branch             = "master"
  }
}
```

Step 3 — Add the `flux_bootstrap_git` resource:

This is what actually installs Flux into the cluster and points it at your repo. Add it after your existing resources:

```
# ------------------------
# Flux Bootstrap
# ------------------------
resource "flux_bootstrap_git" "flux" {
  embedded_manifests = true
  path               = "my-cluster"
  components_extra   = ["image-reflecto

  depends_on = [
    azurerm_kubernetes_cluster.aks,
```

```
    azurerm_role_assignment.ra
  ]
}
```

> **Why `depends_on`?** Flux needs the AKS cluster to exist and the ACR role assignment to be in place before it can bootstrap. The `depends_on` makes Terraform wait for both.

Step 4 — Add the new variable to `variables.tf`:

The `flux` provider needs your GitHub token, which wasn't needed before. Add this variable:

```
variable "gh_token" {
  description = "GitHub personal access
  type        = string
  sensitive   = true
}
```

And add it to your `terraform.tfvars` (which is already in your `.gitignore`):

```
gh_token = "YOUR_GITHUB_PERSONAL_ACCESS
```

The `terraform.tfvars` file should be in the **terraform** directory.

For learning purposes, you should generate a classic token and give it full Read/Write privileges. Give this token an expiry of 60 days.

Apply the Updated Configuration

```
cd terraform
terraform init   # re-run to download t
terraform plan
terraform apply
```

You'll know this setup is complete when you go to your GitHub repo, you should see a new commit created by Flux.
A directory named mycluster should have been created. Run `git pull` to see the changes.

Verify Flux is Running

Run a full health check:

```
flux check
```

You should see all components marked as ✔. Then confirm the pods are up:

```
kubectl get pods -n flux-system
```

All pods should be `Running`. If any are in `CrashLoopBackOff` or `Pending`, check the logs:

```
flux logs --level=error
# or target a specific controller
flux logs --kind=kustomize-controller
```

Check that Flux has picked up your GitRepository source:

```
flux get sources git
```

You should see `flux-system` listed with `Ready = True`.

Subtask 1: Kustomizing your CI Pipeline

In the previous project, you wrote individual YAML manifests for your Tekton pipeline — one for each Task, and one for the Pipeline itself. Right now, Flux has no idea these files exist. To make Flux manage them, you need to bundle them together in a way that both **Kustomize** and **Flux** understand.

Kustomize is a tool built into `kubectl` that lets you manage a collection of Kubernetes manifests as a single unit. You point it at a directory, give it a `kustomization.yaml` that lists the files in that directory, and it treats them all as one deployable package. This is what Flux expects when it syncs a directory from your repo to the cluster.

So the goal of this subtask is: take your existing Tekton manifests, put them in a dedicated `cipipeline/` directory, and add a

`kustomization.yaml` that lists them. Once that's in place, Flux can discover and deploy all of them together — and if you ever add, remove, or change a manifest, Flux will automatically reconcile the cluster to match.

Steps

1. Create the `cipipeline/` directory in your project root:

```
mkdir cipipeline
```

2. Copy your Tekton manifests into it — everything except `pipelinerun.yaml`:

```
cp tekton/kaniko.yaml    cipipeline/
cp tekton/git-clone.yaml cipipeline/
cp tekton/trivy.yaml     cipipeline/
cp tekton/tests.yaml     cipipeline/
cp tekton/pipeline.yaml  cipipeline/
```

> **Why not `pipelinerun.yaml`?** PipelineRuns are one-off execution objects — you trigger them manually. Flux manages the reusable definitions (Tasks, Pipelines), not individual runs.

3. Add a `kustomization.yaml` inside `cipipeline/`:

```
apiVersion: kustomize.config.k8s.io/v1b
kind: Kustomization
resources:
  - kaniko.yaml
  - git-clone.yaml
  - trivy.yaml
  - tests.yaml
  - pipeline.yaml
```

Add Namespaces to All Resources

Flux does not handle the default namespace well, so **every resource** in your manifests needs an explicit namespace. Open each `.yaml` file in the

`cipipeline/` directory and ensure the `metadata` block includes:

```
metadata:
  name: your-resource-name
  namespace: default
```

How Does Flux Know Which Files to Deploy?

Recall that when you bootstrapped Flux, it created a `my-cluster/` directory in your repo. Inside it is a `flux-system/` subdirectory — this contains the core Flux controllers and is managed entirely by Flux itself. You should not manually edit anything in there.

For our own workloads, we need to tell Flux explicitly which directories to watch and deploy. We do this by creating **Flux Kustomization** resources — don't confuse these with regular `kustomization.yaml` files. A Flux Kustomization is a Kubernetes resource that says "watch this path in the repo, and keep the cluster in sync with whatever is there."

To keep things organized, create a `kustomize-sources/` directory inside `my-cluster/`. Any Flux Kustomization resource you drop in here will be automatically picked up by Flux, since it already watches that parent directory. We'll start by creating one for our `cipipeline/` resources:

```
apiVersion: kustomize.toolkit.fluxcd.io
kind: Kustomization
metadata:
  name: cipipeline
  namespace: flux-system
spec:
  interval: 5m
  path: ./cipipeline # this path refers
  prune: true
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Save this as `my-cluster/kustomize-sources/cipipeline.yaml`.

In order to make changes, all you need to do is:

1. Commit changes to the repo.
2. Push.
3. Run `flux reconcile source git flux-system`

You can see the state of everything using `flux get` and the appropriate resource, just like `kubectl` normally does.

Cluster State After Making a Push

```
azureuser@qa-vm:~/cloud-native-handout-
NAME            REVISION
cipipeline      master@sha1:5468d9a5
flux-system     master@sha1:5468d9a5
```

You should see the READY state as True and MESSAGE Applied

Interim Validation

Once your reconcilliation is done, you will realize that your **cipipeline** is working.

At this point, run `kubectl create -f pipelinerun.yaml`, with the pipelinerun file you used last project. Once you have confirmed that:

1. Your pipeline completes without any issues.
2. Your pipeline pushes images to the ACR.

Useful Flux Commands

You'll use these commands extensively in this task. Keep them at your fingertips:

```
flux reconcile source git flux-system #
flux get kustomizations # Use this to l
```

Useful Tekton Commands

You can use these tekton commands to quickly "visualize" your cluster pipeline state

```
kubectl create -f pipelinerun.yaml
tkn pipeline logs -f -L

tkn pipelinerun describe -L
```

Want to verify that Flux is actually deploying your resources? Try a quick sanity check: open `cipipeline/pipeline.yaml` and comment out all tasks except `fetch-source`. Commit and push the change, wait for Flux to reconcile, then inspect your pipeline:

```
kubectl get pipeline
kubectl describe pipeline <your-pipelin
```

You should see only the `fetch-source` task defined. Once you've confirmed Flux is picking up and applying your changes, revert the file back to its original state, commit, push, and move on to Subtask 2.

Subtask 2: Deploy Setup-specific Resources

If you think about what makes your cluster state, you'll know so far, your cluster consists of:

deployments
services
Custom Resource Definitions
NGINX, etc

Anytime you spin your cluster, you painfully have to go through this all over again. The good thing is, you can automate all this using Flux. To make things a bit easier for you, we have come up with kustomizations for you. Please download this task2 handout outside your project root. Somewhere in your home directory:

```
curl https://cloudnativecourse.blob.cor
```

Content of your Handout

```
cicd-task2-handout
├── cdpipeline
│   ├── acrsecret-sealed.yaml
│   ├── image-update-automation.yaml
│   ├── kustomization.yaml
│   ├── llm-image-repository.yaml
│   ├── llm-policy.yaml
│   ├── login-image-repository.yaml
│   └── login-policy.yaml
├── crds
│   ├── kustomization.yaml
│   └── standard-install.yaml
├── helm-releases
│   ├── kustomization.yaml
│   ├── nginx-gateway.yaml
│   └── sealed-secrets.yaml
└── helm-sources
    ├── kustomization.yaml
    ├── nginx-gateway.yaml
    └── sealed-secrets.yaml
```

Your task is straightforward:

> Copy only the folders within cicd-task2-handout and paste them in your project directory root. These directories should be at the same level as your `cipipeline` directory

CRDs

Custom Resource Definitions (CRDs) extend the Kubernetes API by letting you define your own resource types. Out of the box, Kubernetes knows about Pods, Deployments, Services, etc. — but it has no concept of a `Gateway` or `HTTPRoute`. CRDs register those new types with the API server so that Kubernetes can accept, store, and act on them. Without the Gateway API CRDs applied first, any manifest that references a `GatewayClass` or `HTTPRoute` will simply be rejected by the cluster.

The `crds/` directory in your handout contains the **Gateway API standard-channel install** — a single file (`standard-install.yaml`) bundled from the upstream `kubernetes-sigs/gateway-api` project. It registers the `GatewayClass`, `Gateway`,

`HTTPRoute`, and related CRDs that NGINX Gateway Fabric depends on.

To have Flux manage this, add a Flux Kustomization pointing at `./crds` — the same pattern you used for `cipipeline`:

```
apiVersion: kustomize.toolkit.fluxcd.io
kind: Kustomization
metadata:
  name: crds
  namespace: flux-system
spec:
  interval: 5m
  path: ./crds
  prune: true
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Save this as `my-cluster/kustomize-sources/crds.yaml`. Flux will pick it up automatically and apply the Gateway API CRDs to your cluster on the next reconciliation.

> **Order matters:** CRDs must be applied *before* any resources that depend on them. Flux processes Kustomizations in dependency order — if you ever run into timing issues, you can add `dependsOn` to your other Kustomizations to make them wait for `crds` to be ready first.

PRO Tip

Configuring Flux can sometimes be tricky. Whenever you make small changes like adding the CRDs, you should commit and push these to GitHub, run the reconcile command and visualize the state of the cluster before moving on to the next task.

Here's our cluster state after adding the CRDs:

```
azureuser@qa-vm:~/cloud-native-handout-
NAME            REVISION
cipipeline      master@sha1:5749118f
```

```
crds            master@sha1:5749118f
flux-system     master@sha1:5749118f
```

Make sure all listings are TRUE before proceeding

Helm Sources

Before Flux can install a Helm chart, it needs to know *where* to fetch it from. That's what a **HelmRepository** resource does — it registers a Helm chart registry with Flux's source controller, which then periodically polls it for available chart versions.

The `helm-sources/` directory registers two repositories:

- `sealed-secrets` — a standard HTTPS Helm repo hosted by Bitnami Labs, providing the Sealed Secrets controller chart.
- `nginx-gateway-fabric` — an OCI-based registry hosted on GitHub Container Registry (`ghcr.io`), providing the NGINX Gateway Fabric chart.

Both are scoped to the `flux-system` namespace, which is where all Flux source resources live.

To have Flux manage these, add a Flux Kustomization for `helm-sources`:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io
kind: Kustomization
metadata:
  name: helm-sources
  namespace: flux-system
spec:
  interval: 5m
  path: ./helm-sources
  prune: true
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Save this as `my-cluster/kustomize-sources/helm-sources.yaml`.

> **Heads up on ordering:** HelmRepository resources must exist before the HelmRelease resources that reference them. Make sure `helm-sources` is reconciled before `helm-releases` — you'll handle this in the next step.

Remember to use the PRO Tip from before.

NGINX Gateway Fabric Error

When you list all kustomizations, you will encounter an error.
We expect you to fix the following error yourself: helm-sources False False HelmRepository/flux-system/nginx-gateway-fabric dry-run failed: no matches for kind "HelmRepository" in version "source.toolkit.fluxcd.io/v1beta2"

Make sure you fix this before proceeding.

Helm Releases

A **HelmRelease** is how Flux actually installs and manages a Helm chart on your cluster. It references a `HelmRepository` source (defined in the previous step), specifies which chart and version to install, and lets Flux handle upgrades automatically whenever a new matching version is published.

The `helm-releases/` directory defines two releases:

- `sealed-secrets` — installs the Sealed Secrets controller (`sealed-secrets-controller`) from the Bitnami Labs repo. This is what allows you to store encrypted secrets safely in Git. The version constraint `>=1.16.0-0` means Flux will automatically upgrade to newer patch/minor versions as they're published.
- `nginx-gateway-fabric` — installs NGINX Gateway Fabric at a pinned version (`1.2.0`) from the OCI registry. This is the ingress/gateway controller your cluster uses to route external traffic.

To have Flux manage these, add a Flux Kustomization for `helm-releases`:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io
kind: Kustomization
metadata:
  name: helm-releases
  namespace: flux-system
spec:
  interval: 5m
  path: ./helm-releases
  prune: true
  dependsOn:
    - name: helm-sources
    - name: crds
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Save this as `my-cluster/kustomize-sources/helm-releases.yaml`.

> **Note the `dependsOn` field.** This tells Flux to wait until both `helm-sources` and `crds` are fully reconciled before attempting to install these charts. Without it, Flux might try to install NGINX Gateway Fabric before the Gateway API CRDs exist, causing the release to fail.

Remember to use the PRO Tip from before.

Warning

Similarly, you should encounter an error in the helm-release that you should fix before proceeding.

Subtask 3: Continuous Deployment with Flux Image Automation

With CI handled, the final piece is closing the CD loop: when your Tekton pipeline pushes a new image to ACR, Flux should detect it, update the image tag in your deployment manifests, and commit that change back to Git — all automatically.

This is done through three Flux resource types working together: **ImageRepository**, **ImagePolicy**, and **ImageUpdateAutomation**.

ImageRepository tells Flux to watch a specific image in your ACR for new tags. You need one per image:

- `llm-image-repository.yaml` — watches `<YOUR_ACR_NAME>.azurecr.io/llm`
- `login-image-repository.yaml` — watches `<YOUR_ACR_NAME>.azurecr.io/login`

Before using these, replace `<YOUR_ACR_NAME>` with your actual ACR name in both files.

ImagePolicy defines which tag Flux should consider "latest" when multiple tags exist. Both policies use a numerical ascending sort on the timestamp portion of your tag (the pattern `'^(?P<ts>.*)-.*'` strips everything after the first `-`). This means Flux will always select the tag with the highest timestamp — i.e., the most recently built image.

ImageUpdateAutomation is what actually writes the change back to Git. It watches for images whose tags have been updated by an ImagePolicy, edits the relevant file and commits the result to your repo's `master` branch under the `fluxcdbot` author. Flux then picks up that commit on its next reconciliation, completing the loop.

> **Important:** The `image-update-automation.yaml` references `branch: master`.

Sealing your ACR Secret

Flux needs credentials to pull image metadata from your private ACR registry. You cannot store a plain Kubernetes secret in Git — instead, you use **Sealed Secrets** (which you installed via `helm-releases`) to encrypt it first.

The `acrsecret-sealed.yaml` file is a placeholder. You need to generate your own sealed secret and replace `<YOUR_GENERATED_SECRET>` with it. Here's how:

1. Create the raw secret (do not apply it yet):
Go to your ACR on Azure and copy the ACR NAME, USERNAME and PASSWORD. You did this in task 1.

Make sure you run this command in your project root directory.

```
kubectl create secret docker-registry f
  --namespace flux-system \
  --docker-server=<YOUR_ACR_NAME>.azure
  --docker-username=<YOUR_ACR_USERNAME>
  --docker-password=<YOUR_ACR_PASSWORD>
  --dry-run=client -o yaml > acr-secret
```

2. Seal it using the controller that's now running in your cluster:
If you don't have kubeseal installed, use the following command:

```
KUBESEAL_VERSION=$(curl -s https://api.
curl -OL "https://github.com/bitnami-la
tar -xvzf kubeseal-${KUBESEAL_VERSION}-
sudo install -m 755 kubeseal /usr/local
```

```
kubeseal --format=yaml \
  --controller-name=sealed-secrets-cont
  --controller-namespace=flux-system \
  < acr-secret-raw.yaml > cdpipeline/ac
```

3. Delete the raw secret file — never commit it:

```
rm acr-secret-raw.yaml
```

The resulting `acrsecret-sealed.yaml` is safe to commit. Only the Sealed Secrets controller in your cluster can decrypt it.

Marking Your Deployment Manifests for Auto-Update

The automation controller doesn't know which files or lines to update on its own — you have to tell it explicitly using a marker comment placed directly next to the image: line in your deployment manifest. When the controller scans the your repository, it looks for these markers and rewrites only those lines with the latest tag resolved by the referenced policy.

```
image: <YOUR_ACR_NAME>.azurecr.io/llm:l
image: <YOUR_ACR_NAME>.azurecr.io/login
```

1. The deployment manifest for the llm service should be in the **llm_server/k8s** directory
2. The deployment manifest for the login service should be in **django-authentication/k8s** directory

When Flux detects a new tag that satisfies the policy, it rewrites that line in place and commits the update.

Add the Flux Kustomization

```
apiVersion: kustomize.toolkit.fluxcd.io
kind: Kustomization
metadata:
  name: cdpipeline
  namespace: flux-system
spec:
  interval: 5m
  path: ./cdpipeline
  prune: true
  dependsOn:
    - name: cipipeline
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Your cdpipeline should run after the cipipeline.

Save this as `my-cluster/kustomize-sources/cdpipeline.yaml`.

Update your CI Pipeline Kustomization

Update your CI pipeline located at `my-cluster/kustomize-sources/cipipeline.yaml` so that it depends on **helm-releases**. The `dependsOn: helm-releases` ensures the Sealed Secrets controller is running before Flux tries to apply the sealed secret.

Commit and push your changes, visualize your kustomizations and finally, trigger the pipeline:

```
kubectl create -f pipelinerun.yaml
```

Cluster State

Your flux kustomizations should have these resources.

```
azureuser@qa-vm:~/cloud-native-handout-
NAME
helm-controller-58b456999d-7rzsq
image-automation-controller-745594b6d4-
image-reflector-controller-794f4969cb-k
kustomize-controller-76c5d4c759-fjv47
nginx-gateway-fabric-67b46cbd7-7bnp4
notification-controller-6b5b7c8647-sgcz
sealed-secrets-controller-698f9fbbcf-56
source-controller-584b6f58bd-6vtg6

azureuser@qa-vm:~/cloud-native-handout-
NAME            REVISION
cdpipeline      master@sha1:8e2e2177
cipipeline      master@sha1:8e2e2177
crds            master@sha1:8e2e2177
flux-system     master@sha1:8e2e2177
helm-releases   master@sha1:8e2e2177
helm-sources    master@sha1:8e2e2177

azureuser@qa-vm:~/cloud-native-handout-
NAME    LAST SCAN               SUSPEND
llm     2026-03-29T19:01:49Z    False
login   2026-03-29T19:01:49Z    False
```

Directory Tree

The files and folders relevant for this task are:

```
├── cdpipeline
│   ├── acrsecret-sealed.yaml
│   ├── image-update-automation.yaml
│   ├── kustomization.yaml
│   ├── llm-image-repository.yaml
│   ├── llm-policy.yaml
│   ├── login-image-repository.yaml
│   └── login-policy.yaml
```

```
├── cipipeline
│   ├── git-clone.yaml
│   ├── kaniko.yaml
│   ├── kustomization.yaml
│   ├── pipeline.yaml
│   ├── tests.yaml
│   └── trivy.yaml
├── crds
│   ├── kustomization.yaml
│   └── standard-install.yaml
├── helm-releases
│   ├── kustomization.yaml
│   ├── nginx-gateway.yaml
│   └── sealed-secrets.yaml
├── helm-sources
│   ├── kustomization.yaml
│   ├── nginx-gateway.yaml
│   └── sealed-secrets.yaml
├── my-cluster
│   ├── flux-system
│   │   ├── gotk-components.yaml
│   │   ├── gotk-sync.yaml
│   │   └── kustomization.yaml
│   └── kustomize-sources
│       ├── cdpipeline.yaml
│       ├── cipipeline.yaml
│       ├── crds.yaml
│       ├── helm-releases.yaml
│       └── helm-sources.yaml
```

Validation

Navigate to your repository on GitHub.
You should see the last commit made by `fluxcdbot` with the commit message: **Automated image update**

You'll know this task is completed when your deployment.yaml files have been updated by **fluxcdbot**.
Specifically, you should check the container image lines.

Also, make sure your pods are running before making a submission.

Submit your work by running this command:

```
./submitter -t task2
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform